Développement d'un système embarqué pour mesurer et estimer la VO2 max à partir des tests de Léger et Cooper.

Annexe : Code montre Cooper plusieurs fonctions

STN « 2025» _ « 2026 »

Projet : STN21 – Système embarqué VO_2max

Auteur: Ben said Wassim

```
#include <LilyGoLib.h>          // Include LilyGo library for board support (watch hardware)
#include <LV_Helper.h>          // Include LVGL helper library for GUI graphics
#include <time.h>               // Include time library for date/time functions
#include <lvgl.h>               // Include LVGL library for graphical user interface
#include <Preferences.h>        // Include Preferences library for non-volatile storage (EEPROM)
#include <TinyGPS++.h>          // Include TinyGPS++ library for parsing GPS data


// --- Global LVGL Objects ---
lv_obj_t *lbBat, *lbTime;       // Labels for battery percentage and current time
lv_obj_t *label, *btnSend, *btnBack, *labelSending;  // Interface elements for communication
screen
lv_obj_t *splashScreen, *menuScreen, *commScreen, *settingsScreen;  // Main screen objects
lv_obj_t *subScreen;            // Screen object for sub-menus (popup screens)
lv_obj_t *roller1, *roller2, *roller3;  // Roller widgets for time/date selection
lv_obj_t *sliderLumi;           // Slider widget for brightness adjustment

// --- GPS Objects ---
lv_obj_t *labelGPS;             // Label widget for displaying GPS information
lv_timer_t *gpsTimer = NULL;    // Timer for periodic GPS data updates
TinyGPSPlus gps;                // TinyGPS++ object for parsing GPS NMEA sentences
HardwareSerial GPS_Serial(1);   // Hardware serial port UART1 for GPS module (pins 41,42)

// --- Chronometer Objects ---
lv_obj_t *labelChrono;          // Label widget for displaying chronometer time
lv_timer_t * chronoTimer = NULL;  // Timer for updating chronometer display
uint32_t startTime = 0;         // Timestamp when chronometer started (milliseconds)
uint32_t elapsedMillis = 0;     // Elapsed time in milliseconds (accumulated)
bool chronoRunning = false;     // Flag indicating if chronometer is currently running

// --- Cooper Test Objects ---
lv_obj_t *labelCooper;          // Label widget for displaying Cooper test information
lv_obj_t *btnCooperSave;        // Button widget for saving Cooper test results
lv_timer_t *cooperTimer = NULL;   // Timer for Cooper test logic and updates
double prevLat = 0, prevLon = 0;  // Previous latitude and longitude for distance calculation
double distanceTotale = 0;      // Total distance traveled during Cooper test (meters)
bool firstFix = false;          // Flag indicating if first valid GPS position has been recorded
unsigned long cooperStartTime = 0; // Timestamp when Cooper test started (milliseconds)
bool cooperRunning = false;     // Flag indicating if Cooper test is currently running
const unsigned long DUREE_COOPER = 2UL * 60UL * 1000UL; // Test duration: 2 minutes in
milliseconds
bool cooperFixObtained = false;    // Flag indicating if GPS fix has been obtained
bool cooperTestFinished = false;    // Flag indicating if test has finished
bool testTermine = false;          // Flag indicating test is complete (French: test finished)
double vo2Finale = 0;              // Final calculated VO2max value
```

```
// Improved GPS filtering variables
static double latSum = 0, lonSum = 0;   // Sum accumulators for moving average filtering
static int filterCount = 0;             // Counter for number of samples in moving average
const int N_MOY = 10; // Number of points for moving average (increased from 5 to 10 for better smoothing)
const double MIN_DIST = 5.0; // Minimum distance to accept movement (5 meters instead of 2) - filters GPS noise
const double MAX_DIST = 30.0; // Maximum acceptable distance between two points (30 meters) - filters position jumps
const double MAX_SPEED = 7.0; // Maximum realistic speed in m/s (about 18 km/h) - filters unrealistic movements

// GPS quality filter
const int MIN_SATELLITES = 4; // Minimum number of satellites required for reliable position

// Countdown variables
bool cooperCountdown = false;   // Flag indicating countdown is active
int countdownValue = 5;         // Countdown value in seconds (starts at 5)
unsigned long countdownStart = 0; // Timestamp when countdown started

// Distance update interval
unsigned long lastDistanceUpdate = 0; // Timestamp of last distance calculation
const unsigned long DISTANCE_UPDATE_INTERVAL = 5000; // Update distance every 5 seconds

// Position smoothing
double smoothLat = 0, smoothLon = 0; // Smoothed position values using exponential smoothing
const double SMOOTHING_FACTOR = 0.3;  // Smoothing factor (between 0 and 1) - higher = less smoothing

// Rejection of outlier positions
int consecutiveBadReadings = 0;      // Counter for consecutive invalid GPS readings
const int MAX_CONSECUTIVE_BAD = 3;    // Maximum consecutive bad readings before resetting reference position

// Variables for saving final results
double finalDistance = 0;           // Final distance from Cooper test
double finalVO2 = 0;                // Final VO2max value
int finalHeureGPS = 0;              // GPS hour at test completion
int finalMinuteGPS = 0;             // GPS minute at test completion
int finalSecondeGPS = 0;            // GPS second at test completion
int finalJour = 0;                  // Day of test
int finalMois = 0;                  // Month of test
int finalAnnee = 0;                 // Year of test
bool hasCooperData = false;         // Flag indicating if Cooper test data is available

// --- State Variables ---
Preferences prefs;                  // Preferences object for non-volatile storage
uint32_t intervalTime;              // Timer for periodic updates (battery, time)
```

```
uint32_t returnTime = 0;            // Timestamp for returning from LoRa transmission
char buf[64];                       // Buffer for string formatting
bool isSending = false;             // Flag indicating if LoRa transmission is in progress
int tempBrightness = 255;           // Temporary brightness value for preview
bool isDarkTheme = false;           // Flag for dark/light theme (false = light, true = dark)
bool previewDarkTheme = false;      // Preview theme value for settings preview

// Screen enumeration
enum Screen { SCREEN_SPLASH, SCREEN_MENU, SCREEN_COMM, SCREEN_SETTINGS
}; // Available screens
Screen currentScreen = SCREEN_SPLASH; // Current active screen

// --- Function Prototypes ---
void showSplashScreen();            // Display splash/welcome screen
void showMenuScreen();              // Display main menu screen
void showCommScreen();              // Display communication screen (LoRa)
void showSettingsScreen();          // Display settings screen
void updateStatusIndicators();      // Update battery and time display
void createStatusIndicators(lv_obj_t *parent); // Create battery and time labels
void loadLastCooperData();          // Load last saved Cooper test data from preferences
void openGPSScreen();               // Open GPS information screen
void openCooperTest();              // Open Cooper test screen
void openChrono();                  // Open chronometer screen
void openNuitSetting();             // Open night mode settings screen
void openLumiSetting();             // Open brightness settings screen
void openTimeSetting();             // Open time settings screen
void openDateSetting();             // Open date settings screen
void openTempDisplay();             // Open temperature display screen

// --- Memory Management Functions ---
void loadSettings() {
    prefs.begin("watch_cfg", true);     // Open preferences namespace in read-only mode
    isDarkTheme = prefs.getBool("dark", false);  // Load dark theme setting, default false
    tempBrightness = prefs.getInt("lumi", 255);   // Load brightness setting, default 255
    prefs.end();                        // Close preferences namespace

    // Load the last saved Cooper test data
    loadLastCooperData();
}

void loadLastCooperData() {
    prefs.begin("cooper_data", true);      // Open Cooper data namespace in read-only mode
    int testCount = prefs.getInt("test_count", 0);  // Get number of saved tests

    if (testCount > 0) {
        // Load the most recent test (index testCount-1)
        String prefix = "test_" + String(testCount - 1) + "_";  // Create prefix for keys
        finalDistance = prefs.getDouble((prefix + "dist").c_str(), 0);  // Load distance
        finalVO2 = prefs.getDouble((prefix + "vo2").c_str(), 0);        // Load VO2max
```

```
        finalHeureGPS = prefs.getInt((prefix + "heure").c_str(), 0);    // Load hour
        finalMinuteGPS = prefs.getInt((prefix + "minute").c_str(), 0);   // Load minute
        finalSecondeGPS = prefs.getInt((prefix + "seconde").c_str(), 0); // Load second
        finalJour = prefs.getInt((prefix + "jour").c_str(), 0);          // Load day
        finalMois = prefs.getInt((prefix + "mois").c_str(), 0);          // Load month
        finalAnnee = prefs.getInt((prefix + "annee").c_str(), 0);        // Load year
        hasCooperData = true;                // Mark that data is available
    }
    prefs.end();                     // Close preferences namespace
}

void saveCooperResults() {
    prefs.begin("cooper_data", false);     // Open Cooper data namespace in write mode

    // Get number of tests already saved
    int testCount = prefs.getInt("test_count", 0);

    // Save new data with incremental index
    String prefix = "test_" + String(testCount) + "_";  // Create prefix for new test
    prefs.putDouble((prefix + "dist").c_str(), finalDistance);   // Save distance
    prefs.putDouble((prefix + "vo2").c_str(), finalVO2);         // Save VO2max
    prefs.putInt((prefix + "heure").c_str(), finalHeureGPS);     // Save hour
    prefs.putInt((prefix + "minute").c_str(), finalMinuteGPS);   // Save minute
    prefs.putInt((prefix + "seconde").c_str(), finalSecondeGPS); // Save second

    // Save test date
    prefs.putInt((prefix + "jour").c_str(), finalJour);          // Save day
    prefs.putInt((prefix + "mois").c_str(), finalMois);          // Save month
    prefs.putInt((prefix + "annee").c_str(), finalAnnee);        // Save year

    // Increment test counter
    prefs.putInt("test_count", testCount + 1);

    prefs.end();                     // Close preferences namespace

    // Update flag
    hasCooperData = true;
}

// --- Cooper Test Utility Functions ---
double toRad(double deg) {
    return deg * 3.141592653589793 / 180.0;  // Convert degrees to radians for trigonometric
calculations
}

double distanceGPS(double lat1, double lon1, double lat2, double lon2) {
    const double R = 6371000.0; // Earth's radius in meters
    double dLat = toRad(lat2 - lat1);      // Difference in latitude in radians
    double dLon = toRad(lon2 - lon1);      // Difference in longitude in radians
```

```
    lat1 = toRad(lat1);                 // Convert first latitude to radians
    lat2 = toRad(lat2);                 // Convert second latitude to radians

    // Haversine formula for great-circle distance
    double a = sin(dLat/2)*sin(dLat/2) + cos(lat1)*cos(lat2)*sin(dLon/2)*sin(dLon/2);
    double c = 2 * atan2(sqrt(a), sqrt(1-a));  // Central angle in radians

    return R * c;                       // Distance in meters
}

// --- Chronometer Logic ---
void chrono_timer_cb(lv_timer_t * timer) {
    if(chronoRunning) {                 // If chronometer is running
        uint32_t current = millis() - startTime + elapsedMillis; // Calculate total elapsed time
        int ms = (current % 1000) / 10;     // Extract centiseconds (milliseconds/10)
        int sec = (current / 1000) % 60;    // Extract seconds
        int min = (current / 60000);        // Extract minutes
        lv_label_set_text_fmt(labelChrono, "%02d:%02d.%02d", min, sec, ms); // Format as
MM:SS.CC
    }
}

// --- GPS Logic ---
void gps_timer_cb(lv_timer_t * timer) {
    if (labelGPS != NULL) {             // If GPS label exists (screen is active)
        // Read all available GPS data from serial port
        while (GPS_Serial.available() > 0) {
            gps.encode(GPS_Serial.read());  // Feed each byte to TinyGPS++ parser
        }

        if (gps.location.isValid()) {       // If GPS position is valid
            double lat = gps.location.lat();   // Get latitude
            double lon = gps.location.lng();   // Get longitude
            int sats = gps.satellites.value(); // Get number of satellites
            double alt = gps.altitude.meters(); // Get altitude in meters
            double speed = gps.speed.kmph();    // Get speed in km/h

            // Update GPS label with formatted data
            lv_label_set_text_fmt(labelGPS,
                "Latitude: %.6f\n"
                "Longitude: %.6f\n"
                "Altitude: %.1f m\n"
                "Vitesse: %.1f km/h\n"
                "Satellites: %d",
                lat, lon, alt, speed, sats);
        } else {                        // No valid GPS position
            int sats = gps.satellites.value(); // Get visible satellites count
            if (gps.charsProcessed() < 10) {   // If no GPS data received yet
                lv_label_set_text(labelGPS,
```

```
                "Aucune donnee GPS\n\n"
                "Verifiez le module GPS\n"
                "ou la connexion");
            } else {                        // Waiting for signal
                lv_label_set_text_fmt(labelGPS,
                "En attente signal GPS...\n\n"
                "Satellites: %d\n\n"
                "Placez la montre\n"
                "en exterieur", sats);
            }
        }
    }
}

// --- Cooper Test Logic ---
void cooper_timer_cb(lv_timer_t * timer) {
    if (labelCooper != NULL) {              // If Cooper label exists (screen is active)
        // Read GPS data from serial port
        while (GPS_Serial.available() > 0) {
            gps.encode(GPS_Serial.read());   // Feed each byte to TinyGPS++ parser
        }

        // Countdown management
        if (cooperCountdown) {              // If countdown is active
            unsigned long elapsed = millis() - countdownStart;  // Time elapsed since countdown start
            int remaining = 5 - (elapsed / 1000);  // Calculate remaining seconds

            if (remaining > 0) {            // If still counting down
                lv_label_set_text_fmt(labelCooper,
                "DEPART DANS\n\n"
                "%d\n\n"
                "Preparez-vous!",
                remaining);                 // Display countdown number
                return;                     // Exit function - don't process GPS data during countdown
            } else {                        // Countdown finished
                cooperCountdown = false;        // Deactivate countdown
                cooperRunning = true;           // Start the test
                testTermine = false;            // Test not finished yet
                cooperStartTime = millis();      // Record start time
                lastDistanceUpdate = millis();   // Initialize last update time
                distanceTotale = 0;             // Reset total distance
                firstFix = false;               // No first fix yet
                latSum = lonSum = 0;            // Reset filter sums
                filterCount = 0;                // Reset filter counter
                consecutiveBadReadings = 0;      // Reset bad readings counter
            }
        }

        // Check if GPS fix is available
```

```
        if (!gps.location.isValid()) {        // If no GPS fix
            if (!cooperRunning && !cooperCountdown) {  // If test not running
                lv_label_set_text(labelCooper,
                    "RECHERCHE GPS...\n\n"
                    "Sortez dehors\n"
                    "Vue du ciel necessaire");
            }
            return;                           // Exit function - can't proceed without GPS
        }

        // First GPS fix obtained (test not started yet)
        if (!cooperFixObtained && !cooperRunning && !cooperCountdown) {
            cooperFixObtained = true;         // Mark that we have GPS fix
            lv_label_set_text(labelCooper,
                "GPS FIX !\n\n"
                "Appuyez sur START\n"
                "pour commencer");
            return;                           // Exit function - wait for user to press START
        }

        // If the test is currently running
        if (cooperRunning && !testTermine) {
            double lat = gps.location.lat();   // Get current latitude
            double lon = gps.location.lng();   // Get current longitude
            int sat = gps.satellites.isValid() ? gps.satellites.value() : 0; // Get satellite count

            // Check GPS signal quality
            bool isGpsQualityGood = (sat >= MIN_SATELLITES);  // Good quality if enough satellites

            // Calculate distance every 5 seconds
            unsigned long currentTime = millis();
            if (currentTime - lastDistanceUpdate >= DISTANCE_UPDATE_INTERVAL &&
isGpsQualityGood) {
                lastDistanceUpdate = currentTime;  // Update last update time

                // GPS filtering with moving average
                latSum += lat;                // Add to latitude sum
                lonSum += lon;                // Add to longitude sum
                filterCount++;                // Increment sample counter

                if (filterCount >= N_MOY) {      // If we have enough samples for average
                    double avgLat = latSum / filterCount;  // Calculate average latitude
                    double avgLon = lonSum / filterCount;  // Calculate average longitude

                    // Apply exponential smoothing filter
                    if (!firstFix) {              // First fix after test start
                        smoothLat = avgLat;       // Initialize smoothed position
                        smoothLon = avgLon;       // Initialize smoothed position
                        prevLat = avgLat;         // Set previous position
```

```
                prevLon = avgLon;          // Set previous position
                firstFix = true;           // Mark that we have first fix
                consecutiveBadReadings = 0; // Reset bad readings counter
            } else {
                // Exponential smoothing: new = old*(1-factor) + new*factor
                smoothLat = smoothLat * (1 - SMOOTHING_FACTOR) + avgLat *
SMOOTHING_FACTOR;
                smoothLon = smoothLon * (1 - SMOOTHING_FACTOR) + avgLon *
SMOOTHING_FACTOR;

                double d = distanceGPS(prevLat, prevLon, smoothLat, smoothLon); // Calculate
distance

                // Calculate elapsed time and speed
                double timeSeconds = DISTANCE_UPDATE_INTERVAL / 1000.0; // Convert ms
to seconds
                double speed = d / timeSeconds; // Speed in m/s

                // Check if distance is plausible (not too small, not too large, not too fast)
                bool isDistancePlausible = (d >= MIN_DIST && d <= MAX_DIST && speed <=
MAX_SPEED);

                if (isDistancePlausible) {   // Plausible distance - accept it
                    distanceTotale += d;      // Add distance to total
                    prevLat = smoothLat;      // Update previous position
                    prevLon = smoothLon;      // Update previous position
                    consecutiveBadReadings = 0; // Reset bad readings counter

                    // Debug output to serial monitor
                    Serial.print("Distance ajoutée: ");
                    Serial.print(d);
                    Serial.print(" m, Vitesse: ");
                    Serial.print(speed);
                    Serial.println(" m/s");
                } else {                     // Suspect distance - ignore it
                    consecutiveBadReadings++; // Increment bad readings counter
                    Serial.print("Distance ignorée: ");
                    Serial.print(d);
                    Serial.print(" m, Vitesse: ");
                    Serial.print(speed);
                    Serial.println(" m/s (trop grand ou trop petit)");

                    // If too many bad readings consecutively, reset reference position
                    if (consecutiveBadReadings >= MAX_CONSECUTIVE_BAD) {
                        prevLat = smoothLat;   // Reset reference to current position
                        prevLon = smoothLon;   // Reset reference to current position
                        consecutiveBadReadings = 0; // Reset counter
                        Serial.println("Réinitialisation de la position de référence");
                    }
```

```
            }
        }

        latSum = lonSum = 0;          // Reset sums for next averaging period
        filterCount = 0;              // Reset sample counter
    }
}

// Calculate remaining time
unsigned long elapsed = millis() - cooperStartTime;  // Time elapsed since test start
unsigned long remaining = DUREE_COOPER - elapsed;    // Time remaining in test
int minRemain = remaining / 60000;                   // Minutes remaining
int secRemain = (remaining / 1000) % 60;             // Seconds remaining

// Update display with current test status
lv_label_set_text_fmt(labelCooper,
    "DISTANCE: %.0f m\n\n"
    "%02d:%02d\n\n"
    "SAT: %d%s",
    distanceTotale,
    minRemain, secRemain,
    sat,
    (sat < MIN_SATELLITES) ? " (faible)" : "");  // Indicate low satellite count

// Center the text alignment
lv_obj_set_style_text_align(labelCooper, LV_TEXT_ALIGN_CENTER, 0);

// Enlarge font for better visibility of time
static bool policeAgrandie = false;
if (!policeAgrandie) {
    lv_obj_set_style_text_font(labelCooper, &lv_font_montserrat_32, 0);
    policeAgrandie = true;
}

// Check if test is complete
if (elapsed >= DUREE_COOPER && !testTermine) {  // Test duration reached
    testTermine = true;                         // Mark test as finished
    cooperRunning = false;                      // Stop the test
    cooperTestFinished = true;                  // Test completed

    // Save final results
    finalDistance = distanceTotale;             // Store final distance
    vo2Finale = (finalDistance - 504.9) / 44.73; // Calculate VO2max using Cooper formula
    finalVO2 = vo2Finale;                       // Store final VO2max

    // Retrieve GPS time
    int heureGPS = gps.time.isValid() ? gps.time.hour() : 0;      // Get hour
    int minuteGPS = gps.time.isValid() ? gps.time.minute() : 0;   // Get minute
    int secondeGPS = gps.time.isValid() ? gps.time.second() : 0;  // Get second
```

```
            finalHeureGPS = heureGPS;                   // Save hour
            finalMinuteGPS = minuteGPS;                 // Save minute
            finalSecondeGPS = secondeGPS;               // Save second

            // Retrieve date from RTC (Real Time Clock)
            struct tm ti;
            instance.rtc.getDateTime(&ti);              // Read RTC date/time
            finalJour = ti.tm_mday;                     // Save day
            finalMois = ti.tm_mon + 1;                  // Save month (+1 because tm_mon starts at 0)
            finalAnnee = ti.tm_year + 1900;             // Save year (+1900 because tm_year is
years since 1900)

            // IMPORTANT: Save reference to old screen before deleting
            lv_obj_t *oldScreen = subScreen;

            // Create new completion screen
            lv_obj_t *finScreen = lv_obj_create(lv_scr_act());  // Create screen object
            lv_obj_set_size(finScreen, LV_PCT(100), LV_PCT(100)); // Full screen size
            lv_obj_set_style_bg_color(finScreen, isDarkTheme ? lv_color_black() :
lv_color_white(), 0); // Background color based on theme

            // Title label
            lv_obj_t *titreFin = lv_label_create(finScreen);   // Create title label
            lv_label_set_text(titreFin, "TEST TERMINE!");      // "TEST COMPLETED"
            lv_obj_set_style_text_font(titreFin, &lv_font_montserrat_24, 0); // Font size
            lv_obj_set_style_text_color(titreFin, isDarkTheme ? lv_color_white() : lv_color_black(),
0); // Text color
            lv_obj_align(titreFin, LV_ALIGN_TOP_MID, 0, 20);   // Align at top center

            // Distance display
            lv_obj_t *distanceFin = lv_label_create(finScreen); // Create distance label
            lv_label_set_text_fmt(distanceFin, "%.0f m", finalDistance); // Format distance
            lv_obj_set_style_text_font(distanceFin, &lv_font_montserrat_28, 0); // Font size
            lv_obj_set_style_text_color(distanceFin, lv_palette_main(LV_PALETTE_GREEN), 0); //
Green color
            lv_obj_set_style_text_align(distanceFin, LV_TEXT_ALIGN_CENTER, 0); // Center align
            lv_obj_align(distanceFin, LV_ALIGN_CENTER, 0, -30); // Position slightly above center

            // VO2max display
            lv_obj_t *vo2Fin = lv_label_create(finScreen);      // Create VO2max label
            lv_label_set_text_fmt(vo2Fin, "VO2max: %.1f", finalVO2); // Format VO2max
            lv_obj_set_style_text_font(vo2Fin, &lv_font_montserrat_20, 0); // Font size
            lv_obj_set_style_text_color(vo2Fin, lv_palette_main(LV_PALETTE_ORANGE), 0); //
Orange color
            lv_obj_set_style_text_align(vo2Fin, LV_TEXT_ALIGN_CENTER, 0); // Center align
            lv_obj_align(vo2Fin, LV_ALIGN_CENTER, 0, 20);       // Position slightly below center

            // Date and time display
```

```
            lv_obj_t *dateHeure = lv_label_create(finScreen);    // Create date/time label
            lv_label_set_text_fmt(dateHeure, "%02d/%02d/%04d %02d:%02d",
                finalJour, finalMois, finalAnnee,
                finalHeureGPS, finalMinuteGPS);                  // Format date and time
            lv_obj_set_style_text_font(dateHeure, &lv_font_montserrat_14, 0); // Small font
            lv_obj_set_style_text_color(dateHeure, isDarkTheme ? lv_color_white() :
lv_color_black(), 0); // Text color
            lv_obj_set_style_text_align(dateHeure, LV_TEXT_ALIGN_CENTER, 0); // Center align
            lv_obj_align(dateHeure, LV_ALIGN_BOTTOM_MID, 0, -60); // Position at bottom

            // Create container for buttons
            lv_obj_t *btnContainer = lv_obj_create(finScreen);    // Create button container
            lv_obj_set_size(btnContainer, LV_PCT(100), 60);       // Full width, 60px height
            lv_obj_align(btnContainer, LV_ALIGN_BOTTOM_MID, 0, 0); // Position at bottom
            lv_obj_set_style_bg_opa(btnContainer, 0, 0);          // Transparent background
            lv_obj_set_style_border_width(btnContainer, 0, 0);    // No border
            lv_obj_set_flex_flow(btnContainer, LV_FLEX_FLOW_ROW);  // Horizontal layout
            lv_obj_set_flex_align(btnContainer, LV_FLEX_ALIGN_SPACE_EVENLY,
LV_FLEX_ALIGN_CENTER, LV_FLEX_ALIGN_CENTER); // Evenly spaced

            // SAVE button
            lv_obj_t *btnSave = lv_btn_create(btnContainer);      // Create save button
            lv_obj_set_size(btnSave, 100, 45);                    // Button size
            lv_obj_set_style_bg_color(btnSave, lv_palette_main(LV_PALETTE_GREEN), 0); //
Green background
            lv_obj_t *labSave = lv_label_create(btnSave);         // Create button label
            lv_label_set_text(labSave, "SAVE");                   // Button text
            lv_obj_center(labSave);                               // Center label

            lv_obj_add_event_cb(btnSave, [](lv_event_t * e){      // Click event callback
                saveCooperResults();                              // Save results to preferences

                // Show confirmation message
                lv_obj_t *screen = (lv_obj_t*)lv_event_get_user_data(e); // Get screen reference
                lv_obj_t *msg = lv_label_create(screen);          // Create message label
                lv_label_set_text(msg, "Donnees\nsauvegardees!");   // "Data saved!"
                lv_obj_set_style_text_font(msg, &lv_font_montserrat_20, 0); // Font size
                lv_obj_set_style_text_color(msg, lv_palette_main(LV_PALETTE_GREEN), 0); //
Green text
                lv_obj_set_style_text_align(msg, LV_TEXT_ALIGN_CENTER, 0); // Center align
                lv_obj_align(msg, LV_ALIGN_CENTER, 0, 0);         // Position at center

                // Auto-hide message after 2 seconds
                lv_timer_t *timer = lv_timer_create([](lv_timer_t *t){ // Create timer
                    lv_obj_del((lv_obj_t*)lv_timer_get_user_data(t)); // Delete message label
                    lv_timer_del(t);                              // Delete timer
                }, 2000, msg);                                    // 2 seconds delay

            }, LV_EVENT_CLICKED, finScreen);                      // Pass screen to callback
```

```
            // BACK button
            lv_obj_t *btnRetour = lv_btn_create(btnContainer);      // Create back button
            lv_obj_set_size(btnRetour, 100, 45);                    // Button size
            lv_obj_set_style_bg_color(btnRetour, lv_palette_main(LV_PALETTE_BLUE), 0); // Blue
background
            lv_obj_t *labRetour = lv_label_create(btnRetour);       // Create button label
            lv_label_set_text(labRetour, "RETOUR");                 // Button text
            lv_obj_center(labRetour);                               // Center label

            lv_obj_add_event_cb(btnRetour, [](lv_event_t * e){      // Click event callback
                lv_obj_del(lv_scr_act());                           // Delete current screen
                showMenuScreen();                                   // Return to menu screen
            }, LV_EVENT_CLICKED, NULL);                             // No user data

            // Replace old screen with completion screen
            if (oldScreen != NULL) {
                lv_obj_del(oldScreen);                              // Delete old sub-screen
            }
            subScreen = finScreen;                                  // Set new sub-screen

            // Reset flags to prevent recreating completion screen
            cooperFixObtained = false;                              // Reset GPS fix flag

            // IMPORTANT: Pause Cooper timer - no longer needed
            if (cooperTimer != NULL) {
                lv_timer_pause(cooperTimer);                        // Pause timer updates
            }

            // Exit function to avoid further execution
            return;
        }
    }
  }
}

// --- Event Handlers ---
static void cancel_sub_handler(lv_event_t *e) {                     // Cancel/close sub-screen
  if (lv_event_get_code(e) == LV_EVENT_CLICKED) {                   // If button clicked
    instance.setBrightness(tempBrightness);                         // Restore original brightness
    if(chronoTimer) { lv_timer_del(chronoTimer); chronoTimer = NULL; } // Delete chronometer
timer
    if(gpsTimer) { lv_timer_del(gpsTimer); gpsTimer = NULL; }       // Delete GPS timer
    if(cooperTimer) {
      lv_timer_del(cooperTimer);                                    // Delete Cooper timer
      cooperTimer = NULL;
    }
    lv_obj_del(subScreen);                                          // Delete sub-screen
  }
```

```
}

static void cancel_nuit_handler(lv_event_t *e) {                    // Cancel night mode preview
    if (lv_event_get_code(e) == LV_EVENT_CLICKED) {                 // If button clicked
        previewDarkTheme = isDarkTheme;                             // Cancel theme preview
        lv_obj_del(subScreen);                                      // Delete sub-screen
        if (currentScreen == SCREEN_SETTINGS) { lv_obj_del(settingsScreen);
showSettingsScreen(); } // Refresh settings
    }
}

static void save_nuit_handler(lv_event_t *e) {                      // Save night mode setting
    if (lv_event_get_code(e) == LV_EVENT_CLICKED) {                 // If button clicked
        isDarkTheme = previewDarkTheme;                             // Apply theme setting
        prefs.begin("watch_cfg", false);                            // Open preferences in write mode
        prefs.putBool("dark", isDarkTheme);                         // Save theme setting
        prefs.end();                                                // Close preferences
        lv_obj_del(subScreen);                                      // Delete sub-screen
        if (currentScreen == SCREEN_SETTINGS) { lv_obj_del(settingsScreen);
showSettingsScreen(); } // Refresh settings
    }
}

static void save_time_handler(lv_event_t *e) {                      // Save time setting
    if (lv_event_get_code(e) == LV_EVENT_CLICKED) {                 // If button clicked
        struct tm ti; instance.rtc.getDateTime(&ti);                // Read current RTC date/time
        instance.rtc.setDateTime(ti.tm_year + 1900, ti.tm_mon + 1, ti.tm_mday,
                                 lv_roller_get_selected(roller1),
                                 lv_roller_get_selected(roller2), 0);  // Set new time (hours, minutes,
seconds=0)
        lv_obj_del(subScreen);                                      // Delete sub-screen
        updateStatusIndicators();                                   // Update display with new time
    }
}

static void save_date_handler(lv_event_t *e) {                      // Save date setting
    if (lv_event_get_code(e) == LV_EVENT_CLICKED) {                 // If button clicked
        struct tm ti; instance.rtc.getDateTime(&ti);                // Read current RTC date/time
        int year = lv_roller_get_selected(roller1) + 2024;          // Get selected year (2024-2030)
        int month = lv_roller_get_selected(roller2) + 1;            // Get selected month (1-12)
        int day = lv_roller_get_selected(roller3) + 1;              // Get selected day (1-31)
        instance.rtc.setDateTime(year, month, day, ti.tm_hour, ti.tm_min, 0); // Set new date
        lv_obj_del(subScreen);                                      // Delete sub-screen
        updateStatusIndicators();                                   // Update display with new date
    }
}

static void save_cooper_handler(lv_event_t *e) {                    // Save Cooper test results
    if (lv_event_get_code(e) == LV_EVENT_CLICKED) {                 // If button clicked
```

```
        saveCooperResults();                                    // Save results to preferences

        // Display confirmation message
        lv_label_set_text_fmt(labelCooper,
            "DONNEES SAUVEGARDEES!\n\n"
            "Distance: %.0f m\n"
            "VO2max: %.1f\n"
            "Heure: %02d:%02d:%02d\n"
            "Date: %02d/%02d/%04d\n\n"
            "Appuyez RETOUR",
            finalDistance,
            finalVO2,
            finalHeureGPS, finalMinuteGPS, finalSecondeGPS,
            finalJour, finalMois, finalAnnee);                  // Display saved data

        // Hide save button after saving
        lv_obj_add_flag(btnCooperSave, LV_OBJ_FLAG_HIDDEN);     // Hide button
    }
}

// --- Sub-menus ---

void openGPSScreen() {                                          // GPS information screen
    subScreen = lv_obj_create(lv_scr_act());                    // Create sub-screen object
    lv_obj_set_size(subScreen, LV_PCT(100), LV_PCT(100));       // Full screen size
    lv_obj_center(subScreen);                                   // Center the screen
    lv_obj_set_style_bg_color(subScreen, isDarkTheme ? lv_color_black() : lv_color_white(), 0); //
Background color

    lv_obj_t *title = lv_label_create(subScreen);               // Create title label
    lv_label_set_text(title, "GPS INFO");                       // Title text
    lv_obj_set_style_text_font(title, &lv_font_montserrat_24, 0);   // Font size
    lv_obj_set_style_text_color(title, isDarkTheme ? lv_color_white() : lv_color_black(), 0); // Text
color
    lv_obj_align(title, LV_ALIGN_TOP_MID, 0, 10);               // Position at top center


    labelGPS = lv_label_create(subScreen);                      // Create GPS data label
    lv_label_set_text(labelGPS, "Initialisation GPS...");       // Initial message
    lv_obj_set_style_text_color(labelGPS, isDarkTheme ? lv_color_white() : lv_color_black(), 0); //
Text color
    lv_obj_set_style_text_align(labelGPS, LV_TEXT_ALIGN_CENTER, 0);     // Center alignment
    lv_obj_align(labelGPS, LV_ALIGN_CENTER, 0, 0);              // Position at center

    lv_obj_t *btnX = lv_btn_create(subScreen);                  // Create back button
    lv_obj_set_size(btnX, 100, 40);                             // Button size
    lv_obj_align(btnX, LV_ALIGN_BOTTOM_MID, 0, -20);            // Position at bottom
    lv_label_set_text(lv_label_create(btnX), "RETOUR");         // Button text
```

```
    lv_obj_add_event_cb(btnX, cancel_sub_handler, LV_EVENT_CLICKED, NULL); // Click
callback

    if(gpsTimer == NULL) {                                  // If GPS timer doesn't exist
        gpsTimer = lv_timer_create(gps_timer_cb, 1000, NULL);         // Create timer (1 second
interval)
    }
}

void openCooperTest() {                                     // Cooper test screen
    // Reset all Cooper test variables
    cooperRunning = false;                                  // Test not running
    cooperFixObtained = false;                               // No GPS fix obtained
    cooperCountdown = false;                                 // No countdown active
    cooperTestFinished = false;                              // Test not finished
    testTermine = false;                                   // Test not terminated
    distanceTotale = 0;                                    // Reset total distance
    firstFix = false;                                     // No first fix
    latSum = lonSum = 0;                                    // Reset filter sums
    filterCount = 0;                                      // Reset filter counter
    prevLat = prevLon = 0;                                  // Reset previous positions
    countdownValue = 5;                                     // Reset countdown to 5 seconds
    lastDistanceUpdate = 0;                                  // Reset last update time
    consecutiveBadReadings = 0;                                 // Reset bad readings counter

    // Ensure timer is resumed if it was paused
    if (cooperTimer != NULL) {
        lv_timer_resume(cooperTimer);                           // Resume Cooper timer
    }

    subScreen = lv_obj_create(lv_scr_act());                        // Create sub-screen object
    lv_obj_set_size(subScreen, LV_PCT(100), LV_PCT(100));               // Full screen size
    lv_obj_center(subScreen);                                   // Center the screen
    lv_obj_set_style_bg_color(subScreen, isDarkTheme ? lv_color_black() : lv_color_white(), 0); //
Background color

    labelCooper = lv_label_create(subScreen);                       // Create Cooper test label
    lv_label_set_text(labelCooper, "Recherche GPS...");                 // Initial message
    lv_obj_set_style_text_color(labelCooper, isDarkTheme ? lv_color_white() : lv_color_black(), 0);
// Text color
    lv_obj_set_style_text_align(labelCooper, LV_TEXT_ALIGN_CENTER, 0);  // Center alignment
    lv_obj_set_style_text_font(labelCooper, &lv_font_montserrat_18, 0); // Font size
    lv_obj_align(labelCooper, LV_ALIGN_CENTER, 0, -30);                 // Position slightly above
center

    // START button (left side)
    lv_obj_t *btnStart = lv_btn_create(subScreen);                  // Create start button
    lv_obj_set_size(btnStart, 100, 45);                           // Button size
    lv_obj_align(btnStart, LV_ALIGN_BOTTOM_LEFT, 10, -10);              // Position at bottom left
```

```
    lv_obj_t *labStart = lv_label_create(btnStart);                  // Create button label
    lv_label_set_text(labStart, "START");                            // Button text
    lv_obj_center(labStart);                                         // Center label

    lv_obj_add_event_cb(btnStart, [](lv_event_t * e){                // Click event callback
        if (cooperFixObtained && !cooperRunning && !cooperCountdown) { // If GPS ready and test
not running
            // Start countdown
            cooperCountdown = true;                                  // Activate countdown
            countdownStart = millis();                               // Record countdown start time
            lv_obj_t * btn = (lv_obj_t *)lv_event_get_target(e);     // Get button reference
            lv_obj_set_style_bg_color(btn, lv_palette_main(LV_PALETTE_ORANGE), 0); // Change
button color to orange
        }
    }, LV_EVENT_CLICKED, NULL);                                      // Click event

    // SAVE button (hidden initially)
    btnCooperSave = lv_btn_create(subScreen);                        // Create save button
    lv_obj_set_size(btnCooperSave, 100, 45);                         // Button size
    lv_obj_align(btnCooperSave, LV_ALIGN_BOTTOM_MID, 0, -10);        // Position at bottom
center
    lv_obj_t *labSave = lv_label_create(btnCooperSave);              // Create button label
    lv_label_set_text(labSave, "SAVE");                              // Button text
    lv_obj_center(labSave);                                          // Center label
    lv_obj_set_style_bg_color(btnCooperSave, lv_palette_main(LV_PALETTE_GREEN), 0); //
Green background
    lv_obj_add_event_cb(btnCooperSave, save_cooper_handler, LV_EVENT_CLICKED, NULL); //
Click callback
    lv_obj_add_flag(btnCooperSave, LV_OBJ_FLAG_HIDDEN);              // Hide initially

    // BACK button (right side)
    lv_obj_t *btnBack = lv_btn_create(subScreen);                    // Create back button
    lv_obj_set_size(btnBack, 100, 45);                               // Button size
    lv_obj_align(btnBack, LV_ALIGN_BOTTOM_RIGHT, -10, -10);          // Position at bottom
right
    lv_label_set_text(lv_label_create(btnBack), "RETOUR");           // Button text
    lv_obj_add_event_cb(btnBack, cancel_sub_handler, LV_EVENT_CLICKED, NULL); // Click
callback

    if(cooperTimer == NULL) {                                        // If Cooper timer doesn't exist
        cooperTimer = lv_timer_create(cooper_timer_cb, 100, NULL);   // Create timer (100ms
interval)
    }
}

void openChrono() {                                                  // Chronometer screen
    subScreen = lv_obj_create(lv_scr_act());                         // Create sub-screen object
    lv_obj_set_size(subScreen, 220, 180);                            // Fixed size
    lv_obj_center(subScreen);                                        // Center the screen
```

```
    lv_obj_t *title = lv_label_create(subScreen);                    // Create title label
    lv_label_set_text(title, "CHRONO");                              // Title text
    lv_obj_align(title, LV_ALIGN_TOP_MID, 0, -5);                    // Position at top

    labelChrono = lv_label_create(subScreen);                        // Create chronometer display label
    lv_label_set_text(labelChrono, "00:00.00");                      // Initial display
    lv_obj_set_style_text_font(labelChrono, &lv_font_montserrat_32, 0); // Large font
    lv_obj_align(labelChrono, LV_ALIGN_CENTER, 0, -10);              // Position at center

    lv_obj_t *btnStart = lv_btn_create(subScreen);                   // Create start/stop button
    lv_obj_set_size(btnStart, 85, 40);                               // Button size
    lv_obj_align(btnStart, LV_ALIGN_BOTTOM_LEFT, -5, 5);             // Position at bottom left
    lv_obj_t *labStart = lv_label_create(btnStart);                  // Create button label
    lv_label_set_text(labStart, chronoRunning ? "STOP" : "START");   // Dynamic text based on
state
    lv_obj_center(labStart);                                         // Center label

    lv_obj_add_event_cb(btnStart, [](lv_event_t * e){                // Click event callback
        lv_obj_t * btn = (lv_obj_t *)lv_event_get_target(e);         // Get button reference
        lv_obj_t * lab = lv_obj_get_child(btn, 0);                   // Get button label
        if(!chronoRunning) {                                         // If chronometer is stopped
            startTime = millis();                                    // Record start time
            chronoRunning = true;                                    // Start chronometer
            lv_label_set_text(lab, "STOP");                          // Change text to STOP
            lv_obj_set_style_bg_color(btn, lv_palette_main(LV_PALETTE_RED), 0); // Red
background
        } else {                                                     // If chronometer is running
            elapsedMillis += millis() - startTime;                   // Add elapsed time to total
            chronoRunning = false;                                   // Stop chronometer
            lv_label_set_text(lab, "START");                         // Change text to START
            lv_obj_set_style_bg_color(btn, lv_palette_main(LV_PALETTE_GREEN), 0); // Green
background
        }
    }, LV_EVENT_CLICKED, NULL);                                      // Click event

    lv_obj_t *btnReset = lv_btn_create(subScreen);                   // Create reset button
    lv_obj_set_size(btnReset, 85, 40);                               // Button size
    lv_obj_align(btnReset, LV_ALIGN_BOTTOM_RIGHT, 5, 5);             // Position at bottom right
    lv_obj_t *labReset = lv_label_create(btnReset);                  // Create button label
    lv_label_set_text(labReset, "RESET");                            // Button text
    lv_obj_center(labReset);                                         // Center label

    lv_obj_add_event_cb(btnReset, [](lv_event_t * e){                // Click event callback
        chronoRunning = false;                                       // Stop chronometer
        elapsedMillis = 0;                                           // Reset elapsed time
        startTime = millis();                                        // Reset start time
        lv_label_set_text(labelChrono, "00:00.00");                  // Reset display
    }, LV_EVENT_CLICKED, NULL);                                      // Click event
```

```
    lv_obj_t *btnX = lv_btn_create(subScreen);                      // Create close button
    lv_obj_set_size(btnX, 40, 30);                                  // Button size
    lv_obj_align(btnX, LV_ALIGN_TOP_RIGHT, 10, -10);                // Position at top right
    lv_label_set_text(lv_label_create(btnX), "X");                  // Button text
    lv_obj_add_event_cb(btnX, cancel_sub_handler, LV_EVENT_CLICKED, NULL); // Click
callback

    if(chronoTimer == NULL) chronoTimer = lv_timer_create(chrono_timer_cb, 50, NULL); //
Create timer (50ms)
}

void openNuitSetting() {                                            // Night mode settings screen
    subScreen = lv_obj_create(lv_scr_act());                        // Create sub-screen object
    lv_obj_set_size(subScreen, 220, 160);                           // Fixed size
    lv_obj_center(subScreen);                                       // Center the screen
    lv_obj_t *title = lv_label_create(subScreen);                   // Create title label
    lv_label_set_text(title, "MODE NUIT");                          // Title text
    lv_obj_align(title, LV_ALIGN_TOP_MID, 0, 5);                    // Position at top

    lv_obj_t *sw = lv_switch_create(subScreen);                     // Create switch widget
    lv_obj_align(sw, LV_ALIGN_CENTER, 0, -10);                      // Position at center
    if(previewDarkTheme) lv_obj_add_state(sw, LV_STATE_CHECKED);    // Set state based
on preview

    lv_obj_add_event_cb(sw, [](lv_event_t * e){                     // Value change callback
        previewDarkTheme = lv_obj_has_state((lv_obj_t *)lv_event_get_target(e),
LV_STATE_CHECKED); // Update preview
        lv_obj_set_style_bg_color(subScreen, previewDarkTheme ? lv_color_hex(0x333333) :
lv_color_white(), 0); // Change background
    }, LV_EVENT_VALUE_CHANGED, NULL);                               // Value changed event

    lv_obj_t *btnS = lv_btn_create(subScreen);                      // Create save button
    lv_obj_set_size(btnS, 85, 35);                                  // Button size
    lv_obj_align(btnS, LV_ALIGN_BOTTOM_LEFT, -5, 5);                // Position at bottom left
    lv_obj_add_event_cb(btnS, save_nuit_handler, LV_EVENT_CLICKED, NULL); // Click callback
    lv_obj_t *lS = lv_label_create(btnS); lv_label_set_text(lS, "SAUVER"); lv_obj_center(lS); //
Button label

    lv_obj_t *btnC = lv_btn_create(subScreen);                      // Create cancel button
    lv_obj_set_size(btnC, 85, 35);                                  // Button size
    lv_obj_align(btnC, LV_ALIGN_BOTTOM_RIGHT, 5, 5);                // Position at bottom right
    lv_obj_add_event_cb(btnC, cancel_nuit_handler, LV_EVENT_CLICKED, NULL); // Click
callback
    lv_obj_t *lC = lv_label_create(btnC); lv_label_set_text(lC, "RETOUR"); lv_obj_center(lC); //
Button label
}

void openLumiSetting() {                                            // Brightness settings screen
```

```
    subScreen = lv_obj_create(lv_scr_act());                    // Create sub-screen object
    lv_obj_set_size(subScreen, 220, 180);                       // Fixed size
    lv_obj_center(subScreen);                                   // Center the screen
    sliderLumi = lv_slider_create(subScreen);                   // Create slider widget
    lv_obj_set_size(sliderLumi, 160, 20);                       // Slider size
    lv_obj_center(sliderLumi);                                  // Center the slider
    lv_slider_set_range(sliderLumi, 10, 255);                   // Set range (10-255)
    lv_slider_set_value(sliderLumi, tempBrightness, LV_ANIM_OFF);       // Set initial value
    lv_obj_add_event_cb(sliderLumi, [](lv_event_t* e){              // Value change callback
        instance.setBrightness(lv_slider_get_value((lv_obj_t*)lv_event_get_target(e))); // Apply
brightness immediately
    }, LV_EVENT_VALUE_CHANGED, NULL);                           // Value changed event
    lv_obj_t *btnS = lv_btn_create(subScreen);                  // Create OK button
    lv_obj_set_size(btnS, 85, 35); lv_obj_align(btnS, LV_ALIGN_BOTTOM_LEFT, -5, 10); //
Position at bottom left
    lv_obj_add_event_cb(btnS, [](lv_event_t* e){                // Click callback
        tempBrightness = lv_slider_get_value(sliderLumi);           // Save brightness value
        prefs.begin("watch_cfg", false); prefs.putInt("lumi", tempBrightness); prefs.end(); // Save to
preferences
        lv_obj_del(subScreen);                                  // Close sub-screen
    }, LV_EVENT_CLICKED, NULL);                                 // Click event
    lv_label_set_text(lv_label_create(btnS), "OK");             // Button text
    lv_obj_t *btnC = lv_btn_create(subScreen);                  // Create cancel button
    lv_obj_set_size(btnC, 85, 35); lv_obj_align(btnC, LV_ALIGN_BOTTOM_RIGHT, 5, 10); //
Position at bottom right
    lv_obj_add_event_cb(btnC, cancel_sub_handler, LV_EVENT_CLICKED, NULL); // Click
callback
    lv_label_set_text(lv_label_create(btnC), "RETOUR");         // Button text
}

void openTimeSetting() {                                        // Time settings screen
    subScreen = lv_obj_create(lv_scr_act());                    // Create sub-screen object
    lv_obj_set_size(subScreen, 220, 200);                       // Fixed size
    lv_obj_center(subScreen);                                   // Center the screen
    roller1 = lv_roller_create(subScreen);                      // Create hour roller
    lv_roller_set_options(roller1,
"00\n01\n02\n03\n04\n05\n06\n07\n08\n09\n10\n11\n12\n13\n14\n15\n16\n17\n18\n19\n20\n21\n
22\n23", LV_ROLLER_MODE_NORMAL); // Options 0-23
    lv_obj_set_size(roller1, 70, 90); lv_obj_align(roller1, LV_ALIGN_CENTER, -40, 0); // Size and
position
    roller2 = lv_roller_create(subScreen);                      // Create minute roller
    lv_roller_set_options(roller2,
"00\n01\n02\n03\n04\n05\n06\n07\n08\n09\n10\n11\n12\n13\n14\n15\n16\n17\n18\n19\n20\n21\n
22\n23\n24\n25\n26\n27\n28\n29\n30\n31\n32\n33\n34\n35\n36\n37\n38\n39\n40\n41\n42\n43\n
44\n45\n46\n47\n48\n49\n50\n51\n52\n53\n54\n55\n56\n57\n58\n59",
LV_ROLLER_MODE_NORMAL); // Options 0-59
    lv_obj_set_size(roller2, 70, 90); lv_obj_align(roller2, LV_ALIGN_CENTER, 40, 0); // Size and
position
    lv_obj_t *btnS = lv_btn_create(subScreen);                  // Create OK button
```

```
    lv_obj_set_size(btnS, 85, 35); lv_obj_align(btnS, LV_ALIGN_BOTTOM_LEFT, -5, 10); // Position at bottom left
    lv_obj_add_event_cb(btnS, save_time_handler, LV_EVENT_CLICKED, NULL); // Click callback
    lv_label_set_text(lv_label_create(btnS), "OK");                    // Button text
    lv_obj_t *btnC = lv_btn_create(subScreen);                         // Create cancel button
    lv_obj_set_size(btnC, 85, 35); lv_obj_align(btnC, LV_ALIGN_BOTTOM_RIGHT, 5, 10); // Position at bottom right
    lv_obj_add_event_cb(btnC, cancel_sub_handler, LV_EVENT_CLICKED, NULL); // Click callback
    lv_label_set_text(lv_label_create(btnC), "RETOUR");                // Button text
}

void openDateSetting() {                                               // Date settings screen
    subScreen = lv_obj_create(lv_scr_act());                           // Create sub-screen object
    lv_obj_set_size(subScreen, 230, 200);                              // Fixed size
    lv_obj_center(subScreen);                                          // Center the screen
    roller1 = lv_roller_create(subScreen);                             // Create year roller
    lv_roller_set_options(roller1, "2024\n2025\n2026\n2027\n2028\n2029\n2030", LV_ROLLER_MODE_NORMAL); // Options 2024-2030
    lv_obj_set_size(roller1, 65, 80); lv_obj_align(roller1, LV_ALIGN_CENTER, -70, 0); // Size and position
    roller2 = lv_roller_create(subScreen);                             // Create month roller
    lv_roller_set_options(roller2, "01\n02\n03\n04\n05\n06\n07\n08\n09\n10\n11\n12", LV_ROLLER_MODE_NORMAL); // Options 1-12
    lv_obj_set_size(roller2, 50, 80); lv_obj_align(roller2, LV_ALIGN_CENTER, 0, 0); // Size and position
    roller3 = lv_roller_create(subScreen);                             // Create day roller
    lv_roller_set_options(roller3, "01\n02\n03\n04\n05\n06\n07\n08\n09\n10\n11\n12\n13\n14\n15\n16\n17\n18\n19\n20\n21\n22\n23\n24\n25\n26\n27\n28\n29\n30\n31", LV_ROLLER_MODE_NORMAL); // Options 1-31
    lv_obj_set_size(roller3, 50, 80); lv_obj_align(roller3, LV_ALIGN_CENTER, 60, 0); // Size and position
    lv_obj_t *btnS = lv_btn_create(subScreen);                         // Create OK button
    lv_obj_set_size(btnS, 85, 35); lv_obj_align(btnS, LV_ALIGN_BOTTOM_LEFT, -5, 10); // Position at bottom left
    lv_obj_add_event_cb(btnS, save_date_handler, LV_EVENT_CLICKED, NULL); // Click callback
    lv_label_set_text(lv_label_create(btnS), "OK");                    // Button text
    lv_obj_t *btnC = lv_btn_create(subScreen);                         // Create cancel button
    lv_obj_set_size(btnC, 85, 35); lv_obj_align(btnC, LV_ALIGN_BOTTOM_RIGHT, 5, 10); // Position at bottom right
    lv_obj_add_event_cb(btnC, cancel_sub_handler, LV_EVENT_CLICKED, NULL); // Click callback
    lv_label_set_text(lv_label_create(btnC), "RETOUR");                // Button text
}

void openTempDisplay() {                                               // Temperature display screen
    subScreen = lv_obj_create(lv_scr_act());                           // Create sub-screen object
```

```
    lv_obj_set_size(subScreen, 200, 150);                    // Fixed size
    lv_obj_center(subScreen);                                // Center the screen
    lv_obj_t *t = lv_label_create(subScreen);                // Create temperature label
    lv_label_set_text_fmt(t, "TEMP PUCE:\n%.1f C", instance.pmu.getTemperature()); // Display
chip temperature
    lv_obj_set_style_text_align(t, LV_TEXT_ALIGN_CENTER, 0);      // Center alignment
    lv_obj_center(t);                                        // Center the label
    lv_obj_t *btn = lv_btn_create(subScreen);                // Create back button
    lv_obj_set_size(btn, 100, 40); lv_obj_align(btn, LV_ALIGN_BOTTOM_MID, 0, 0); // Position at
bottom
    lv_obj_add_event_cb(btn, cancel_sub_handler, LV_EVENT_CLICKED, NULL); // Click
callback
    lv_label_set_text(lv_label_create(btn), "RETOUR");           // Button text
}

// --- Basic Functions ---
void updateStatusIndicators() {                              // Update battery and time display
    if (lbBat != NULL && lbTime != NULL) {
        int bat = instance.pmu.getBatteryPercent();          // Read battery percentage
        lv_label_set_text_fmt(lbBat, "%d%% %s", bat, instance.pmu.isCharging() ? "+" : ""); //
Display battery with charging indicator
        struct tm ti;
        instance.rtc.getDateTime(&ti);                       // Read current time from RTC
        strftime(buf, 64, "%H:%M", &ti);                     // Format time as HH:MM
        lv_label_set_text(lbTime, buf);                      // Display time
        lv_color_t textColor = isDarkTheme ? lv_color_white() : lv_color_black(); // Text color based
on theme
        lv_obj_set_style_text_color(lbTime, textColor, 0);       // Apply color to time label
        lv_obj_set_style_text_color(lbBat, textColor, 0);        // Apply color to battery label
    }
}

void createStatusIndicators(lv_obj_t *parent) {              // Create status indicator labels
    lv_obj_set_style_bg_color(parent, isDarkTheme ? lv_color_black() : lv_color_white(), 0); // Set
screen background
    lbTime = lv_label_create(parent);                        // Create time label
    lv_obj_align(lbTime, LV_ALIGN_TOP_LEFT, 10, 5);          // Position at top left
    lv_obj_set_style_text_font(lbTime, &lv_font_montserrat_14, 0);  // Small font
    lv_obj_set_style_text_color(lbTime, isDarkTheme ? lv_color_white() : lv_color_black(), 0); //
Text color

    lbBat = lv_label_create(parent);                         // Create battery label
    lv_obj_align(lbBat, LV_ALIGN_TOP_RIGHT, -10, 5);         // Position at top right
    lv_obj_set_style_text_font(lbBat, &lv_font_montserrat_14, 0);   // Small font
    lv_obj_set_style_text_color(lbBat, isDarkTheme ? lv_color_white() : lv_color_black(), 0); // Text
color

    updateStatusIndicators();                                // Update with current values
}
```

```
static void settings_grid_handler(lv_event_t *e) {                  // Settings grid button handler
    int id = (intptr_t)lv_event_get_user_data(e);                   // Get button ID from user data
    if (lv_event_get_code(e) == LV_EVENT_CLICKED) {                 // If button clicked
        if (id == 0) openTimeSetting();                             // ID 0: Time settings
        else if (id == 1) openDateSetting();                        // ID 1: Date settings
        else if (id == 2) openTempDisplay();                        // ID 2: Temperature display
        else if (id == 3) openLumiSetting();                        // ID 3: Brightness settings
        else if (id == 4) openNuitSetting();                        // ID 4: Night mode settings
        else if (id == 5) openChrono();                             // ID 5: Chronometer
        else {                                                      // Other buttons (6,7)
            subScreen = lv_obj_create(lv_scr_act());                // Create sub-screen
            lv_obj_set_size(subScreen, 180, 100); lv_obj_center(subScreen); // Size and center
            lv_obj_t *l = lv_label_create(subScreen);               // Create label
            lv_label_set_text_fmt(l, "Bouton %d\nLibre", id + 1); lv_obj_center(l); // Placeholder text
            lv_obj_add_event_cb(subScreen, cancel_sub_handler, LV_EVENT_CLICKED, NULL); // Close on click
        }
    }
}

// --- Navigation Screens ---
void showSplashScreen() {                                           // Splash/welcome screen
    currentScreen = SCREEN_SPLASH;                                  // Set current screen
    splashScreen = lv_obj_create(lv_scr_act());                     // Create splash screen object
    lv_obj_set_size(splashScreen, LV_PCT(100), LV_PCT(100));        // Full screen size
    createStatusIndicators(splashScreen);                           // Add status indicators

    lv_obj_t *welcome = lv_label_create(splashScreen);              // Create welcome label
    lv_label_set_text(welcome, "Bonjour\nWassim");                  // Personalized welcome message
    lv_obj_set_style_text_font(welcome, &lv_font_montserrat_32, 0);     // Large font
    lv_obj_set_style_text_color(welcome, isDarkTheme ? lv_color_white() : lv_color_black(), 0); // Text color
    lv_obj_align(welcome, LV_ALIGN_CENTER, 0, -30);                 // Position above center

    lv_obj_t *btn = lv_btn_create(splashScreen);                    // Create menu button
    lv_obj_align(btn, LV_ALIGN_CENTER, 0, 50);                      // Position below center
    lv_obj_add_event_cb(btn, [](lv_event_t*e){ lv_obj_del(splashScreen); showMenuScreen(); }, LV_EVENT_CLICKED, NULL); // Click callback
    lv_label_set_text(lv_label_create(btn), "MENU");                // Button text
}

// Drop-down menu with centered buttons
void showMenuScreen() {                                             // Main menu screen
    currentScreen = SCREEN_MENU;                                    // Set current screen
    menuScreen = lv_obj_create(lv_scr_act());                       // Create menu screen object
    lv_obj_set_size(menuScreen, LV_PCT(100), LV_PCT(100));          // Full screen size
    createStatusIndicators(menuScreen);                             // Add status indicators
```

```
    // Create container for buttons starting after status indicators
    lv_obj_t *panel = lv_obj_create(menuScreen);                    // Create panel for buttons
    lv_obj_set_size(panel, LV_PCT(100), LV_PCT(85));                // Panel size (85% of screen)
    lv_obj_align(panel, LV_ALIGN_TOP_LEFT, 0, 30); // Position 30px from top to leave room for
time display

    // Make panel transparent and borderless
    lv_obj_set_style_bg_opa(panel, 0, 0);                           // Transparent background
    lv_obj_set_style_border_width(panel, 0, 0);                     // No border
    lv_obj_set_style_pad_all(panel, 0, 0);                          // No padding

    // Enable vertical scrolling
    lv_obj_set_scrollbar_mode(panel, LV_SCROLLBAR_MODE_AUTO);       // Auto scrollbar
    lv_obj_set_scroll_dir(panel, LV_DIR_VER);                       // Vertical scrolling only

    // Add 4 buttons to scrollable panel
    const char* opts[] = {"LORA", "REGLAGES", "GPS", "COOPER"};     // Menu options
    int btnHeight = 50;                                             // Button height
    int btnSpacing = 15; // Spacing between buttons

    for(int i = 0; i < 4; i++) {
        lv_obj_t *btn = lv_btn_create(panel);                       // Create button
        lv_obj_set_width(btn, 180);                                 // Fixed width
        lv_obj_set_height(btn, btnHeight);                          // Fixed height

        // Center buttons horizontally
        lv_obj_align(btn, LV_ALIGN_TOP_MID, 0, i * (btnHeight + btnSpacing) + 10); // Vertical
positioning

        // Add shadow for better visibility
        lv_obj_set_style_shadow_width(btn, 8, 0);                   // Shadow effect
        lv_obj_set_style_shadow_ofs_y(btn, 3, 0);                   // Shadow offset

        // Event handler
        lv_obj_add_event_cb(btn, [](lv_event_t* e){                 // Click callback
            int id = (intptr_t)lv_event_get_user_data(e);           // Get button ID
            if(id == 0) {                                           // ID 0: LORA
                lv_obj_del(menuScreen);                             // Delete menu screen
                showCommScreen();                                   // Show communication screen
            }
            else if(id == 1) {                                      // ID 1: SETTINGS
                lv_obj_del(menuScreen);                             // Delete menu screen
                showSettingsScreen();                               // Show settings screen
            }
            else if(id == 2) {                                      // ID 2: GPS
                openGPSScreen();                                    // Open GPS screen
            }
            else if(id == 3) {                                      // ID 3: COOPER
```

```
            openCooperTest();                                   // Open Cooper test screen
        }
    }, LV_EVENT_CLICKED, (void*)(intptr_t)i);                   // Pass ID as user data

    // Add button text
    lv_obj_t *label = lv_label_create(btn);                     // Create button label
    lv_label_set_text(label, opts[i]);                          // Set text
    lv_obj_center(label);                                       // Center label
}

// Adjust panel height to enable scrolling
int totalHeight = 4 * (btnHeight + btnSpacing) + 30;            // Calculate total height
lv_obj_set_height(panel, totalHeight > 220 ? 220 : totalHeight);    // Set panel height (max
220px)
}

void showCommScreen() {                                         // Communication screen (LoRa)
    currentScreen = SCREEN_COMM;                                // Set current screen
    commScreen = lv_obj_create(lv_scr_act());                   // Create communication screen
    lv_obj_set_size(commScreen, LV_PCT(100), LV_PCT(100));      // Full screen size
    createStatusIndicators(commScreen);                         // Add status indicators

    label = lv_label_create(commScreen);                        // Create info label

    // Display last Cooper test data if available
    if (hasCooperData) {                                        // If data available
        lv_label_set_text_fmt(label,
            "DERNIER TEST COOPER:\n\n"
            "Distance: %.0f m\n"
            "VO2max: %.1f\n"
            "Heure: %02d:%02d:%02d\n"
            "Date: %02d/%02d/%04d",
            finalDistance, finalVO2,
            finalHeureGPS, finalMinuteGPS, finalSecondeGPS,
            finalJour, finalMois, finalAnnee);                  // Display saved data
    } else {                                                    // No data available
        lv_label_set_text(label, "Aucune donnee Cooper\ndisponible");   // Error message
    }

    lv_obj_set_style_text_color(label, isDarkTheme ? lv_color_white() : lv_color_black(), 0); // Text
color
    lv_obj_set_style_text_align(label, LV_TEXT_ALIGN_CENTER, 0);    // Center alignment
    lv_obj_align(label, LV_ALIGN_CENTER, 0, -40);               // Position above center

    btnSend = lv_btn_create(commScreen);                        // Create send button
    lv_obj_set_size(btnSend, 100, 45);                          // Button size
    lv_obj_align(btnSend, LV_ALIGN_CENTER, -55, 40);            // Position left of center
    lv_obj_add_event_cb(btnSend, [](lv_event_t*e){              // Click callback
        isSending = true;                                       // Mark sending in progress
```

```
        lv_obj_add_flag(label, LV_OBJ_FLAG_HIDDEN);                    // Hide info label
        lv_obj_clear_flag(labelSending, LV_OBJ_FLAG_HIDDEN);           // Show sending label

        // Build LoRa message with Cooper data
        if (hasCooperData) {                                           // If data available
            char loraMsg[256];
            snprintf(loraMsg, sizeof(loraMsg),
                "COOPER|%.0f|%.1f|%02d:%02d:%02d|%02d/%02d/%04d",
                finalDistance, finalVO2,
                finalHeureGPS, finalMinuteGPS, finalSecondeGPS,
                finalJour, finalMois, finalAnnee);                     // Format message

            lv_label_set_text_fmt(labelSending, "ENVOI...\n\n%s", loraMsg); // Display sending
message
            radio.transmit(loraMsg);                                   // Transmit via LoRa
        } else {                                                       // No data available
            lv_label_set_text(labelSending, "ENVOI...\n\nAucune donnee");  // Display error
            radio.transmit("COOPER|NO_DATA");                          // Send default message
        }

        returnTime = millis() + 2000;                                  // Set return time (2 seconds)
    }, LV_EVENT_CLICKED, NULL);                                        // Click event
    lv_label_set_text(lv_label_create(btnSend), "ENVOI");              // Button text

    btnBack = lv_btn_create(commScreen);                               // Create back button
    lv_obj_set_size(btnBack, 100, 45);                                 // Button size
    lv_obj_align(btnBack, LV_ALIGN_CENTER, 55, 40);                    // Position right of center
    lv_obj_add_event_cb(btnBack, [](lv_event_t*e){ lv_obj_del(commScreen); showMenuScreen();
}, LV_EVENT_CLICKED, NULL); // Click callback
    lv_label_set_text(lv_label_create(btnBack), "RETOUR");             // Button text

    labelSending = lv_label_create(commScreen);                        // Create sending status label
    lv_obj_set_style_text_color(labelSending, isDarkTheme ? lv_color_white() : lv_color_black(),
0); // Text color
    lv_obj_set_style_text_align(labelSending, LV_TEXT_ALIGN_CENTER, 0);    // Center
alignment
    lv_obj_center(labelSending);                                       // Center the label
    lv_obj_add_flag(labelSending, LV_OBJ_FLAG_HIDDEN);                 // Hidden initially
}

void showSettingsScreen() {                                            // Settings screen
    currentScreen = SCREEN_SETTINGS;                                   // Set current screen

    settingsScreen = lv_obj_create(lv_scr_act());                      // Create settings screen object
    lv_obj_set_size(settingsScreen, LV_PCT(100), LV_PCT(100));         // Full screen size
    createStatusIndicators(settingsScreen);                            // Add status indicators

    lv_obj_t *cont = lv_obj_create(settingsScreen);                    // Create button container
    lv_obj_set_size(cont, 240, 180);                                   // Container size
```

```
    lv_obj_align(cont, LV_ALIGN_BOTTOM_MID, 0, 0);                    // Position at bottom
    lv_obj_set_flex_flow(cont, LV_FLEX_FLOW_ROW_WRAP);                // Flexible row wrap
layout
    lv_obj_set_flex_align(cont, LV_FLEX_ALIGN_CENTER, LV_FLEX_ALIGN_CENTER,
LV_FLEX_ALIGN_CENTER); // Center alignment
    lv_obj_set_style_bg_opa(cont, 0, 0);                              // Transparent background
    lv_obj_set_style_border_width(cont, 0, 0);                        // No border

    const char* names[] = {"Heure", "Date", "Temp", "Lumi", "Nuit", "Chrono", "7", "8"}; // Button
names

    for(int i = 0; i < 8; i++) {
        lv_obj_t *btn = lv_btn_create(cont);                          // Create button
        lv_obj_set_size(btn, 65, 40);                                 // Button size
        lv_obj_add_event_cb(btn, settings_grid_handler, LV_EVENT_CLICKED, (void*)(intptr_t)i); //
Click callback

        if(i == 4 && previewDarkTheme) {                              // Night mode button with preview
            lv_obj_set_style_bg_color(btn, lv_palette_main(LV_PALETTE_BLUE), 0); // Special color
for preview
        }

        lv_obj_t *lbl = lv_label_create(btn);                         // Create button label
        lv_label_set_text(lbl, names[i]);                             // Set button text
        lv_obj_center(lbl);                                           // Center label
    }

    // Back button at top
    lv_obj_t *br = lv_btn_create(settingsScreen);                     // Create back button
    lv_obj_set_size(br, 40, 30);                                      // Button size
    lv_obj_align(br, LV_ALIGN_TOP_MID, 0, 0);                         // Position at top center

    lv_obj_add_event_cb(br, [](lv_event_t *e){                        // Click callback
        lv_obj_del(settingsScreen);                                   // Delete settings screen
        showMenuScreen();                                             // Return to menu
    }, LV_EVENT_CLICKED, NULL);                                       // Click event

    lv_obj_t *lbl = lv_label_create(br);                              // Create label
    lv_label_set_text(lbl, "X");                                      // Text X
    lv_obj_center(lbl);                                               // Center label
}

void setup() {
    Serial.begin(115200);                                             // Initialize serial communication at
115200 baud

    instance.begin();                                                 // Initialize LilyGo watch instance
    beginLvglHelper(instance);                                        // Initialize LVGL helper
    instance.pmu.enableBattDetection();                               // Enable battery detection
```

```
    // FULL POWER ENABLE for GPS module
    instance.pmu.setALDO3Voltage(3300);                    // Set ALDO3 voltage to 3.3V
    instance.pmu.enableALDO3();                            // Enable ALDO3 (GPS power)
    instance.pmu.setALDO4Voltage(3300);                    // Set ALDO4 voltage to 3.3V
    instance.pmu.enableALDO4();                            // Enable ALDO4
    instance.pmu.setBLDO1Voltage(3300);                    // Set BLDO1 voltage to 3.3V
    instance.pmu.enableBLDO1();                            // Enable BLDO1

    // GPS INITIALIZATION
    GPS_Serial.begin(38400, SERIAL_8N1, 41, 42);           // Initialize GPS serial (baud
rate 38400, pins 41(RX), 42(TX))

    loadSettings();                                        // Load saved settings from preferences

    radio.setFrequency(868.0);                             // Set LoRa frequency to 868 MHz

    showSplashScreen();                                    // Display splash screen
    instance.setBrightness(tempBrightness);                // Apply saved brightness setting

    intervalTime = millis();                               // Initialize interval timer
}

void loop() {
    instance.loop();                                       // Run LilyGo instance loop (handles
hardware)
    lv_timer_handler();                                    // Handle LVGL timers (GUI updates)

    // Handle return after LoRa transmission
    if (isSending && millis() > returnTime) {              // If sending finished
        lv_obj_clear_flag(label, LV_OBJ_FLAG_HIDDEN);      // Show info label
        lv_obj_add_flag(labelSending, LV_OBJ_FLAG_HIDDEN); // Hide sending label
        isSending = false;                                 // Reset sending flag
    }

    // Update battery and time every 30 seconds
    if (millis() > intervalTime) {                         // Timer reached
        updateStatusIndicators();                          // Update status displays
        intervalTime = millis() + 30000;                   // Set next update time (30 seconds)
    }
}
```